Warm up

Last precept, we saw how to declare a function pointer, for instance:

```
int (*cmp)(const void *, const void *);
```

What do you think is the syntax for creating an array `cmps` of these function pointers?

(A)
```
int *cmps[](const void *, const void *);
```
(B)
```
int (*cmps)(const void *, const void *)[];
```
(C)
```
int (*cmps[])(const void *, const void *);
```
(D)
```
int *(cmps[])(const void *, const void *);
```

- (C) is the correct answer
- (A), (B) and (D) are all illegal declarations
- We will see how to decipher this today!

Precept 13: Declarations, definitions, scope, linkage and duration

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 19, 2026

Overview

Declarations vs. definitions

- A *declaration* introduces an identifier to the compiler
- A *definition* actually creates the entity being declared, which causes memory to be allocated for the identifier
  - All definitions are declarations (but the reverse is not true)

Scope, linkage and storage duration

- *Scope*: the region of the program where an identifier is visible
  - File: visible throughout the entire source file
  - Block: visible only within a block (i.e., between pair of curly braces)
  - Also: function scope and function prototype scope (out of scope, no pun intended)
- *Linkage*: whether an identifier can be referred to from other translation units (i.e., source files)
  - External (`extern`): accessible from other files (default)
  - Internal (`static`): only accessible within the file it is declared
- *Storage duration*: how long the memory associated with an identifier exists during program execution
  - Automatic / temporary: memory is allocated when the block is entered and deallocated when the block is exited; stored on the runtime stack
  - Static / process: memory is allocated when the program starts and deallocated when the program ends; stored in the data or BSS segment

Function declarations and definitions

- Function declaration: informs the compiler of the function's signature (i.e., name, return type, parameters) and linkage
  - Has no body and does not need to specify parameter names
- Function definition: provides the implementation of the function, which causes compiler to generate machine code for the function (stored in the text segment)
  - Has a body and must specify parameter names

Code	What	Linkage
`int f1(int);`	declaration	external
`extern int f2(int);`	declaration	external
`static int f3(int);`	declaration	internal
`int f4(int i){ ... }`	definition	external
`extern int f5(int i){ ... }`	definition	external
`static int f6(int i){ ... }`	definition	internal

Declaring variables with process duration

- File-scope variables have process (static storage) duration
  - Note: file-scope variables with static *storage duration* can be declared with the `static` keyword (internal linkage) or without it (external linkage)
- Variables with static storage duration are allocated when the program starts and deallocated when the program ends; stored in the data or BSS segment
  - If explicitly initialised to non-zero value → data segment
    - `int i = 5; /*data section */`
  - If not explicitly initialised → BSS (block started by symbol) segment
    - `int j; /*bss section; initialized to 0 */`
  - If explicitly initialised to zero → compiler-dependent (BSS on armlab)
    - `int k = 0; /*bss section with gcc217 */`

Putting it together: scope, linkage and duration

Code	What	Scope	Linkage	Duration	Where
`int a = 5;`	definition	file	external	process	data
`int b;`	definition	file	external	process	bss
~~`extern int c = 5;`~~	definition	file	external	process	data
`extern int d;`	declaration	file	external	process	???
`static int e = 5;`	definition	file	internal	process	data
`static int f;`	definition	file	internal	process	bss
`void fun(int g){`	definition	block	internal	temporary	stack
`  int h = 5;`	definition	block	internal	temporary	stack
`  int i;`	definition	block	internal	temporary	stack
~~`  extern int j = 5;`~~	ILLEGAL				
~~`  extern int k;`~~	declaration	block	???[1]	process	???
`  static int l = 5;`	definition	block	internal	process	data
`  static int m;`	definition	block	internal	process	bss
`  ... }`					

See *C Variable Declarations and Definitions* handout for footnotes

[1] Linkage is inherited from file-scope declaration of `k`, undefined if no such declaration

The `static` keyword

- We have seen two ways to use the `static` keyword:
  - To declare an identifier with internal linkage (i.e., only accessible within the file it is declared)
    - ◇ When applied to a global variable or function (file scope)
  - To declare a process-duration variable (i.e., allocated when the program starts and deallocated when the program ends)
    - ◇ When applied to a local variable (block scope)
- Note: both of these differ from Java's `static` keyword (which is used to create class variables and methods that are shared across all instances of the class)

Global variable declarations and definitions

- See *C Variable Declarations and Definitions* handout for examples
- General observations
  - Across all files, an externally-linked symbol can have at most one definition (but multiple declarations fine)
  - `static` creates an internally-linked symbol, which shadows any externally-linked symbol with the same name in other files
  - When using `extern` declaration, at least one *definition* must be provided in some file
- Advice: minimise use of global (file scope) variables!
  - When you do use global variables, try to make them `static` (and `const` when possible)
  - Global variables with external linkage can be a nightmare to maintain and debug
  - Code that uses global variables is not thread-safe (requires synchronisation)

Overview

Deciphering declarations: the right-left rule

- Read as follows:
  - `*` as "pointer to" (always on the left side)
  - `[]` as "array of" (always on the right side)
  - `()` as "function returning" (always on the right side)
- To decipher a declaration:
  - Step 0: get rid of array size and parameters
  - Step 1: find the identifier being declared
  - Step 2: read to the right until run out of symbols or hit a right paren
  - Step 3: read to the left until run out of symbols or hit a left paren
  - Step 4: repeat steps 2 and 3 until run out of symbols
  - Step 5: reintroduce array size and parameters
- Warning: the right-left rule does not determine if a declaration is legal
- More details and examples in *Complex C Declarations* handout
- `cdecl`: a tool available on armlab that can be used to decipher complex declarations
  - Also available at `https://cdecl.org/`

Example: `int *p[]`

```
int *p[];
```

Apply the right-left rule:

1. Find the identifier: `p` $\Longrightarrow$ "p is"
2. Move right: find `[]` $\Longrightarrow$ "p is array of"
3. Move left: find `*` $\Longrightarrow$ "p is array of pointer to"
4. Keep going left: find `int` $\Longrightarrow$ "p is array of pointer to int"

Thus: `p` is an array where each element is a pointer to `int`

Your turn!

```
int x[5];
```

declare `x` as array of 5 `int`s

```
int x(int i);
```

declare `x` as function (`int`) returning `int`

```
int *x();
```

declare `x` as function returning pointer to `int`

```
int *x[];
```

declare `x` as array of pointer to `int`

```
int (*x)();
```

declare `x` as pointer to function returning `int`

Overview

Summary

- A more formal treatment of declarations and definitions
- How scope, linkage and storage duration affect the visibility and lifetime of identifiers
- Using the right-left rule to decipher complex declarations
- Dates to remember:
  - Assignment 3: due Tue, 3/24 at 9pm
  - Assignment 3 Quiz: Wed, 3/25 in lecture